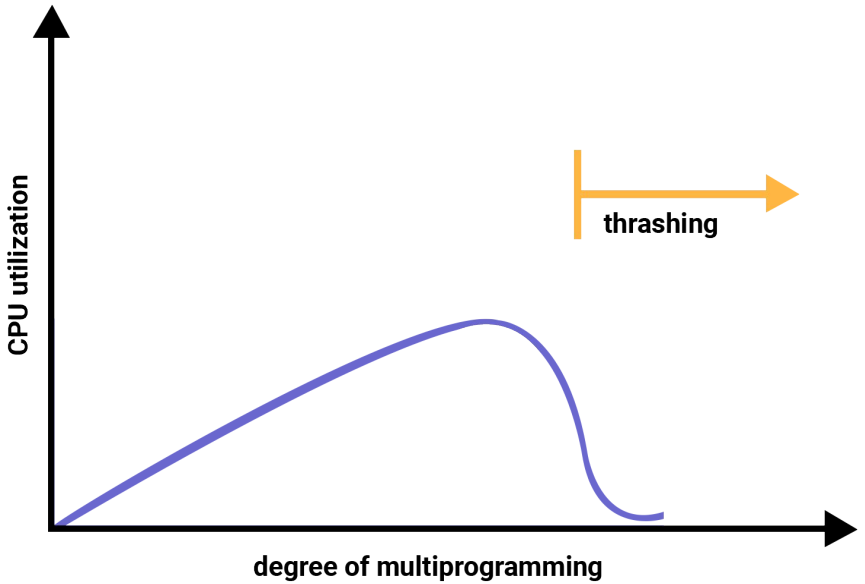
Thrashing and Virtual Memory Conclusion

Dr. Jit Mukherjee

Thrashing

- Intermediate CPU Scheduling - number of frames allocated to a low-priority process goes below minimum number -> Suspend, free remaining pages.
- Any process, !enough frames -> Page fault-> Local Replacement, All active pages -> More page fault
- Thrashing - If a process spending more time paging than executing
- Causes of Thrashing - OS monitor CPU utilization -> If CPU utilization low => introduce new process, increase degree of multiprogramming
- Global replacement -> removes any
- A process needs more frames
  - Page faults
  - Takes frames -> other processes
  - Those process more page fault
  - Faulting process queue up in paging device -> decrease CPU utilization
- OS sees decrease CPU utilization, introduce new process

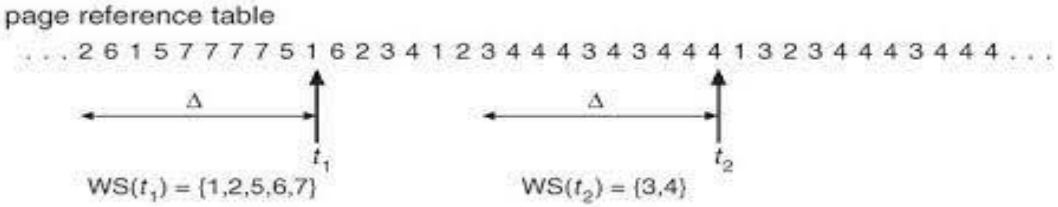


Cause of Thrashing

- New process takes frames from running processes -> More page fault, longer queue in paging device
  - CPU utilization drops further -> CPU scheduler increases degree of multiprogramming
- Thrashing has occurred -> System throughput plunges, effective memory access time increases rapidly -> No work done effectively, only paging
- As the degree of multiprogramming increase, CPU utilization increases -> Reaches a maximum -> Thrashing -> CPU utilization decreases
  - Must decrease the degree of multiprogramming
- **Local Replacement Algorithm** - limit the effects
  - If one process starts thrashing it can not steal frames from other -> !cascade thrashing
  - But effective memory access time still increase. Not fully solved
- Provide a process as many frames as it needs -> Locality Model
  - As a process executes it moves locality to locality. A functions calls it defines it's locality. Depends upon the program structure.
  - Allocate enough frame to accomodate the locality. It will not fault until change of locality
  - Fewer frames than size of current locality -> Thrashing

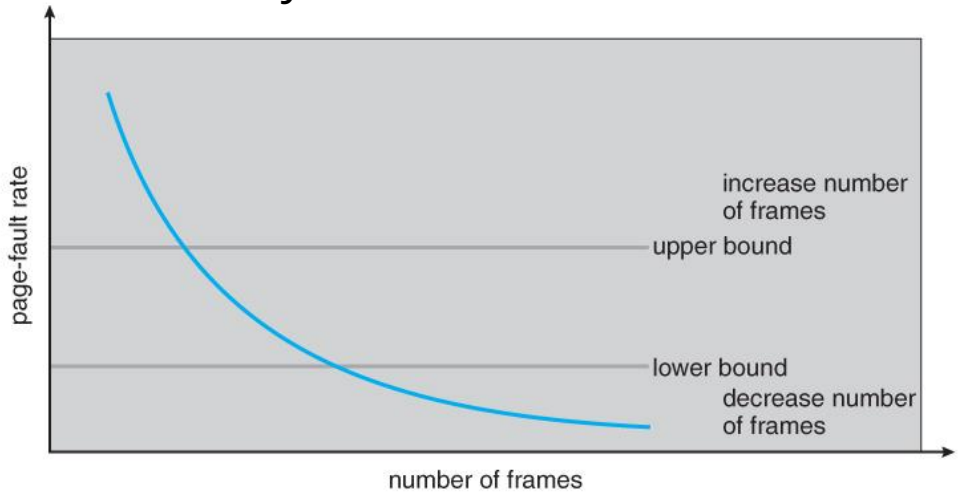
Working Set Model

- Based on assumption of locality. Δ -> **Working set window** -> examine most recent Δ page references -> The set of pages is **Working Set**
  - If a page is active it will be in working set otherwise not.
  - Working Set -> Approximation of programs locality
- Δ = 10, t1={1,2,5,6,7} and changed to t2={3,4}
  - Accuracy depends on Δ. If it is too small -> not entire locality, big ->overlaped localities
- D = $\sum$ WSS_i . WSS_i -> working set size for each process. D -> total demand
  - If D>m => number of available frames -> Thrashing will occur
- Monitors Δ of each process -> allocates to working set enough frames -> If enough extra frame Initiate another process -> Optimize CPU Utilization
  - Id D>m -> Suspend one process -> Swap ll the pages



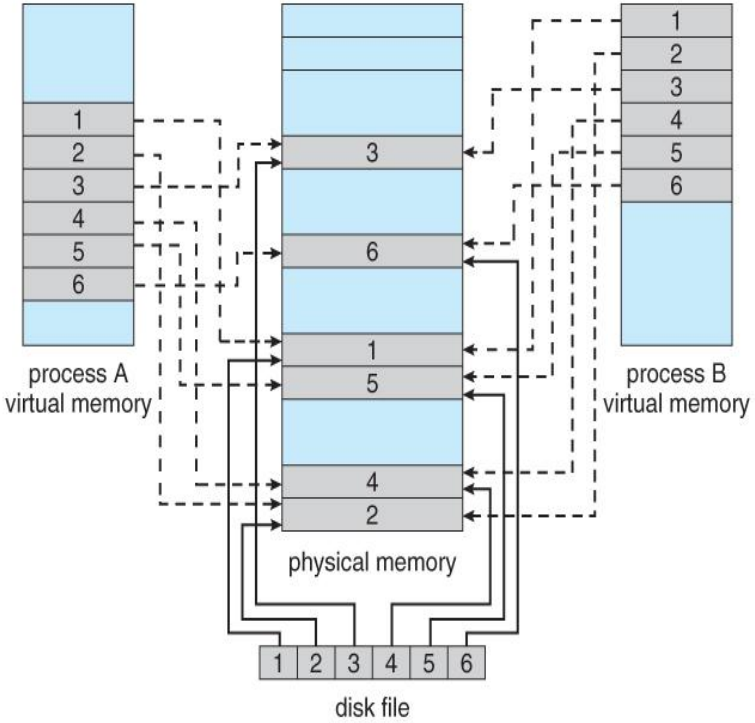
Page-Fault Frequency

- Knowledge of working set can be useful for prepaging. Page-fault frequency has a direct approach -> Thrashing has high page fault rate
  - Too high -> process need more frames; too low->process may have too many frames
  - Upper bound and Lower bound. Page fault rate > upper bound -> allocate process another frame. Page fault rate < lower bound -> remove frame frame from process
  - Page fault rate increases and no frame->May suspend process. Free frame->high page fault
- there is a direct relationship between the page-fault rate and the working-set, as a process moves from one locality to another:



Memory-Mapped FIles

- Rather than accessing data files directly via the file system with every file access, data files can be paged into memory, much faster accesses (except when page-faults occur.) This is known as *memory-mapping* a file.
- Mechanism
  - A file is mapped within a process's virtual address space, and then paged in as needed using the ordinary demand paging system.
  - file writes are made to the memory page frames, and are not immediately written out to disk. (Purpose of "flush( )" system call)
  - Important to "close( )" a file when one is done writing to it ->data can be safely flushed out to disk and memory frames freed up.
  - Some systems provide special system calls to memory map files and use direct disk access. Other systems map the file to process address space and map the file to kernel address space otherwise, but do memory mapping in either case.
  - File sharing is made possible by mapping the same file to the address space of more than one process, Copy-on-write is supported, and mutual exclusion techniques may be needed to avoid synchronization problems.



Allocating Kernel Memory

- Additional memory allocated to the kernel-> cannot be so easily paged.
  - Some of it, I/O buffering, direct access by devices, must be contiguous and not by paging.
  - Other memory is used for internal kernel data structures of various sizes, and is often locked (restricted from swapped out), management of this resource difficult.
- Buddy System
  - The Buddy *System* allocates memory using a *power of two allocator*.
  - block of correct size -> not available -> splitting the next larger block in two matched buddies.
    - if that larger size is not available, then the next largest available size is split, and so on.
  - Free lists are maintained for every size block.
  - If the necessary block size is not available upon request, a free block from the next largest size is split recursively into two buddies of the desired size.
  - When a block is freed, its buddy's address is calculated, and the free list for that size block is checked to see if the buddy is also free. If it is, then the two buddies are coalesced into one larger free block.

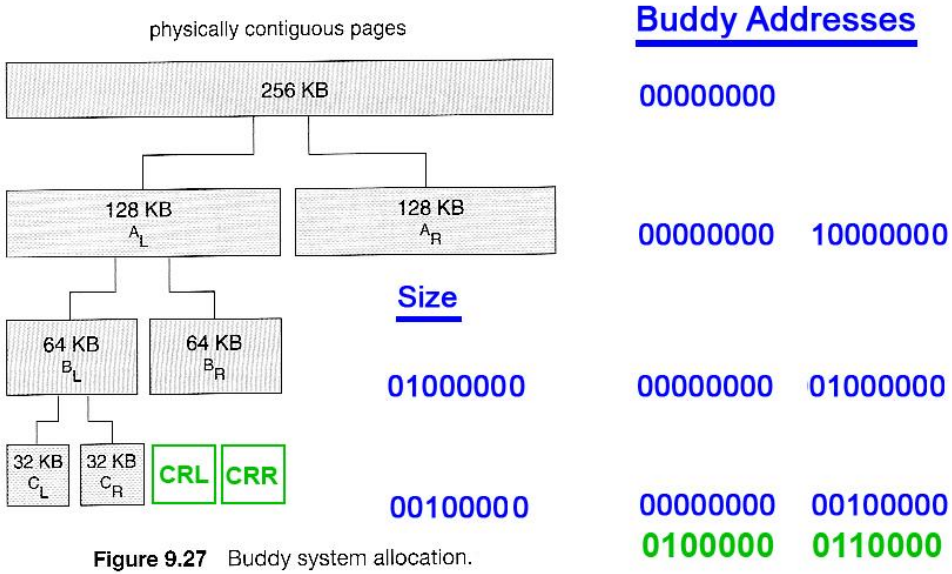
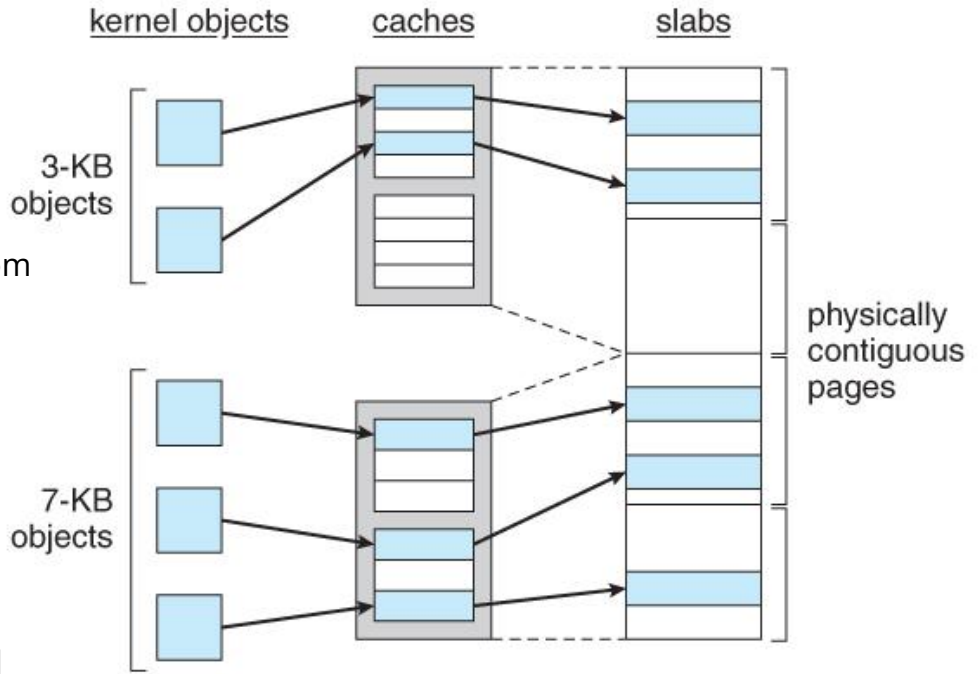


Figure 9.27 Buddy system allocation.

Slab Allocation

- *Slab Allocation* allocates memory to the kernel in chunks -> *slabs* -> consisting of one or more contiguous pages.
  - kernel creates separate caches for each type of data structure it might need from one or more slabs.
  - Initially the caches are marked empty, and are marked full as they are used.
- New requests for space in the cache is first granted from empty or partially empty slabs
  - if all slabs are full, then additional slabs are allocated.
- Benefits of slab allocation-> lack of internal fragmentation
  - fast allocation of space for individual structures
- Solaris uses slab allocation for the kernel and also for certain user-mode memory allocations.
- Linux used the buddy system prior to 2.2 and switched to slab allocation since then.



Prepaging and Page Size

- Prepaging
  - The basic idea behind *prepaging* is to predict the pages that will be needed in the near future
    - page them in before they are actually requested.
  - If a process was swapped out and we know -> its working set at the time
    - when we swap it back in -> page back in the entire working set, before the page faults occur.
  - With small (data) files we can go ahead and prepage all of the pages at one time.
  - Prepaging can be of benefit if the prediction is good and the pages are needed eventually, but slows the system down if the prediction is wrong.
- Page Size
  - There are quite a few trade-offs of small versus large page sizes:
    - Small pages waste less memory due to internal fragmentation.
    - Large pages require smaller page tables.
  - For disk access, the latency and seek times greatly outweigh the actual data transfer times.
    - Much faster to transfer one large page of data than two or more smaller pages containing the same amount of data.
  - Smaller pages match locality better, because we are not bringing in data that is not really needed.
    - Small pages generate more page faults, with attending overhead.
  - The physical hardware may also play a part in determining page size.
  - It is hard to determine an "optimal" page size for any given system. Current norms range from 4K to 4M, and tend towards larger page sizes as time passes.

TLB Reach and Inverted Page Tables

- TLB Reach
  - *D*efined as the amount of memory that can be reached by the pages listed in the TLB.
  - Ideally the working set would fit within the reach of the TLB.
  - Increasing the size of the TLB is an obvious way of increasing TLB reach, but TLB memory is very expensive and also draws lots of power.
  - Increasing page sizes increases TLB reach, but also leads to increased fragmentation loss.
  - Some systems provide multiple size pages to increase TLB reach while keeping fragmentation low.
    - Multiple page sizes requires that the TLB be managed by software, not hardware.
- Inverted Page Tables
  - Stores one entry for each frame instead of one entry for each virtual page.
    - This reduces the memory requirement for the page table
    - Loose the information needed to implement virtual memory paging.
  - Solution -> keep a separate page table for each process, for virtual memory management.
    - These are kept on disk, and only paged in when a page fault occurs.
    - They are not referenced with every memory access the way a traditional page table.

Programme Structure and I/O Interlock

- Programme Structure
  - A pair of nested loops to access elements in a 1024 x 1024 2Dl array (32-bit ints)
  - Arrays in C -> row-major order, each row would occupy a page of memory.
  - If the loops are nested
    - Outer, inner loop increments the row and column
    - An entire row can be processed before the next page fault,
      - Yielding 1024 page faults total.
  - On the other hand, if the loops are nested the other way the program worked down the columns instead of across the rows
    - every access would be to a different page, yielding a new page fault for each access, or over a million page faults all together.
  - Be aware that different languages store their arrays differently. FORTRAN for example stores arrays in column-major format instead of row-major.
- I/O Interlock and Page Locking
  - it may be desirable to *lock* pages in memory, and not let them get paged out:
    - Certain kernel operations cannot tolerate having their pages swapped out.
  - If an I/O controller is doing direct-memory access, it would be wrong to change pages in the middle of the I/O operation.
  - In a priority based scheduling system, low priority jobs may need to wait
    - there is a danger of their pages being paged out before they get a chance to use them even once after paging them in.
    - In this situation pages may be locked when they are paged in, until the process that requested them gets at least one turn in the CPU.